

双发射乱序超标量 LoongArch32 处理器设计报告

2026 全国大学生计算机系统能力培养大赛
CPU 设计赛（龙芯杯）团体赛

设计版本：v2.10（func 58/58 双环境回归通过）

2026 年 8 月

目录

1. 设计目标与总体方案
 2. 核心设计哲学：选择性前递与串行点
 3. 关键模块设计
 4. 性能优化迭代史
 5. 频率设计与验证
 6. 正确性验证方法学
 7. 参考与声明
- 附录 A. 综合与 LUT 深度报告

1. 设计目标与总体方案

本设计实现一款支持 LoongArch32 精简版指令集（LA32R）的双发射乱序超标量处理器核，目标是在通过全部 58 个功能测试点的前提下，最大化性能测试得分。设计遵循 "面积/频率预算集中在大概率事件，罕见事件退回串行"的工程哲学，最终以 IPC 的微小让步换取主频裕量与验证收敛速度。

1.1 微架构总览

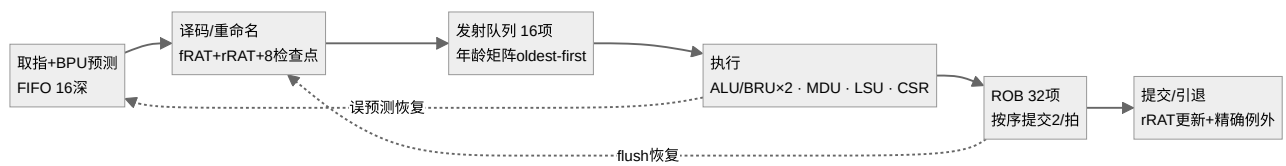


图 1 处理器流水线总体组织

表 1 微架构关键参数

参数	配置	参数	配置
取指宽度	4 条/拍（16B 对齐块）	ROB	32 项环形（2分配/4写回/2提交）
分派/发射	2 条/拍	发射队列	16 项集中式，年龄矩阵 oldest-first
写回宽度	4 口（lane0/lane1/MDU/LSU）	物理寄存器	64 项（32 架构 + 32 自由）
提交宽度	2 条/拍	投机恢复	8 深重命名检查点，一拍回滚
分支预测	BTB128 行粒度 + GShare + RAS8	Store Buffer	4 深，精确 store → load 序检查
ICache	16KB 2 路，四行突发 refill	DCache	8KB 直映射，write-through
乘法器	33×33 单乘法器两级流水（3拍）	除法器	迭代式（34 拍），不阻塞 ALU

1.2 流水线组织

前端（取指 + BPU 预测 → 16 深取指 FIFO）→ 译码/重命名（fRAT 投机表 + rRAT 引退表 + checkpoint）→ 发射（IQ 年龄矩阵选最老就绪 uop）→ 寄存器堆读 → 执行（ALU/BRU×2、MDU、LSU、CSR）→ 写回（4 口）→ ROB 按序提交（例外/中断在头部处理，保证精确例外）。

2. 核心设计哲学：选择性前递与串行点

超标量设计中，旁路/前递网络提升 IPC 但拉长组合逻辑、压缩主频。本设计的基本 权衡原则：只 为绝大多数常见情况设计前递；罕见情况不做前递，退回串行执行。该原则落实为三层：

第一层：执行结果前递全覆盖。4 个写回口的结果当拍前递到 IQ 唤醒与操作数采集，背靠背 ALU 依赖零气泡——这是约 99% 依赖的场景，必须快。

第二层：load 路径有限前递。仅前递 store buffer 命中数据；DCache miss 的 load 不投机唤醒后续消费者，按 miss 完成重发——罕见路径不占关键路径。

第三层：串行点集中管理。CSR 读写、ll.w/sc.w、syscall/break/ertn 等 特权/原子操作标记为 serial，分派后封锁后续分派直至 ROB 排空。这些指令在测试负载中占比极低（停顿画像 serial 类仅占 4.1%），完全串行化的代价可忽略，换来的 是这些路径上**零旁路、零投机状态修复逻辑**。

2.1 jirl 去串行化 (v2.10)

jirl（间接跳转）在 v1 中作为简化也走串行点。v2.10 分析发现 BTB+RAS 已能完整预测 jirl（ret 弹 RAS、call 压栈、其余走 BTB 目标），预测错误时走与条件分支完全相同的 bru_flush + checkpoint 恢复路径，遂将其移出串行点。该改动使 serial 停顿下降 38% (20,004 → 12,339 拍)，总拍数再降 1.05%，代价仅是 checkpoint 占用 429 → 495 次（容量充裕）。

设计教训：串行点清单需要随微架构能力演进而复审——v1 的保守简化在 BPU 能力补齐后成为纯损失。每一次“串行化”决策都应附有“何时可以移除”的判定条件。

3. 关键模块设计

3.1 分支预测器 (BPU v3)

- **BTB 128 项直接映射，行粒度** (index=pc[10:4], tag=pc[31:11])：一行记录该 16B 块内分支的槽位偏移、类别 (COND/CALL/RET/UNCOND) 与目标地址。
- **GShare PHT**：全局历史 XOR PC 索引两位饱和计数器，仅条件分支查 PHT。
- **RAS 8 深**：call 预测拍压栈（返回地址 = 分支 pc+4），ret 预测拍弹栈给出目标；空栈禁止预测（行粒度单槽会被同行后写覆盖，防止脏数据误导）。
- 预测在取指拍完成，预测正确的 taken 分支**零 flush**——v3 最大的单项收益之一。
- 更新时机：BRU 发射时以**解析出的真实目标与类别**写 BTB/PHT，投机路径上的 jirl 也只写入真实目标，无脏数据。

3.2 重命名与投机恢复

- fRAT（投机映射表）+ rRAT（引退映射表）双表结构；提交时 rRAT 更新，flush 时 fRAT 从 checkpoint 一拍恢复。

- **8 深分支检查点**：每个投机分支快照 fRAT (32×6bit) + freelist head + ROB tail，误预测一拍恢复，无需逐项回滚。
- **freelist 例外回滚 O(1)**：例外检测拍组合统计 ROB 窗口内写目的项数 (flush_wen_cnt)，下一拍 freelist head 一次回滚到位；同拍在飞分配的 pd 消耗 一并计入，杜绝物理寄存器泄漏 (该泄漏曾致定时器中断约 30 次后 freelist 耗尽 死锁，已在设计中根除并有回归用例)。

3.3 发射队列与执行单元

- 16 项集中式 IQ，每拍选两个最老的就绪 uop 发射 (年龄矩阵，分配序号比较)。oldest-first 保证串行点判定 (须位于 ROB 头部) 语义简单。
- lane0 兼作 BRU：分支在发射拍解析，误预测当拍发起 flush。
- MDU：33×33 有符号/无符号复用单乘法器，两级流水 (操作数拍 + 乘积拍)，连续乘 3 拍/条；除法迭代 34 拍，busy 期间不阻塞 ALU lane。
- **两阶段 ROB 分配**：rename 的 uop 内容寄存器输出 (T+1 有效)，alloc/pdold 组合输出 (T 有效)，T 沿推进 tail、T+1 沿按 uop 自带 ROB 标签写项——消除 分派 → 分配的组合级联；配套"新槽控制位即拍清零"防止旧代残留造成伪提交。

3.4 存储子系统

- **DCache 8KB 直映射 write-through + no-write-allocate**：比赛负载写局部性差，WT 免除一致性/脏行管理；store 经 4 深 store buffer 吸收。
- **Store → Load 精确序**：IQ 发射 load 前以 ROB 标签环形距离比较 SB 各项，仅当 SB 中存在更老 store 才阻塞 load，误阻塞极少。
- **ICache 16KB 2 路组相联**，v2.8 四行 refill 为当前形态：
 - miss 且 +63B 范围全 cached 时发起 64B 突发 (arlen=15)，一次取 4 行；后台行 (第 2/3/4 行) 在 beat3 拍锁存基址后独立计数写入，不占用 miss 主流程状态机；
 - **S_WAIT 出口 recheck 复查拍**：后台填期间产生的新 miss 在 S_WAIT 排队，后台 burst 完成后先复查 tag——很大概率已被后台行覆盖，命中即省一次完整 burst；
 - 效果：命中率 69% → 83%，miss 事件 16,363 → 9,133 (-44%)。
- **cached 窗口**：0x1c (SRAM) 与 0x0 (DDR3 数据段) 均 cached；uncached 请求 (confreg/UART 等 MMIO) 走单字通道，按地址区间自动判定。

3.5 前端与取指 FIFO

- 每拍取一个 16B 块 (4 条)，跨块边界由 skip/first_low 机制处理 (非对齐 redirect 目标的首块裁剪，同时判定 ADEF)。
- **取指 FIFO 16 深**，解耦取指与分派；余量预留机制：当前 FIFO 占用 + 在途块 4 条 ≤ 12 才发新块请求。

- **resp_kill 门控** (v2.7 关键修复)：预测转向后在途旧流请求尚未消化（杀单挂起）时禁发新请求。否则在途计数使新请求地址跳块、pc 标签仍按旧块打，数据/标签错位装入 FIFO，取到全零触发 INE 异常。该缺陷在 8 深版本靠紧余量 意外免疫，加深后必须显式封堵。

4. 性能优化迭代史

全部优化均经 verilator 双环境（lean 裸 SRAM + 完整 SoC，含 AXI 随机延迟）58/58 功能点回归，并以"停顿画像"探针（分派停顿按 ROB 满 / IQ 近满 / 前端饥饿 / 串行 / freelist 空五类分解）定位瓶颈后数据驱动决策，杜绝凭直觉改动。

表 2 性能迭代史（lean 环境总拍数，越低越好）

版本	改动	lean 拍数	单版收益
v2.0 基线	双发射乱序 + DCache 上线	384,779	—
v2.1	ICache 链式顺序预取	—	—
v2.2	MDU 乘法流水化（33×33 单乘法器两级）	—	—
v3	BPU 上线（BTB128 + GShare + RAS8）	—	taken 分支免 flush
v2.3/2.4	ICache 16KB 2 路 + 双行 refill + 后台填	—	-9.8%
v2.5	cached 窗口扩至 DDR3 数据段	376,6xx	-2.13%
v2.7	取指 FIFO 8 → 16 + resp_kill 门控修复	316,790	-1.95%
v2.8	ICache 四行 refill + S_WAIT recheck	304,467	-3.9%
v2.9	IQ 12 → 16（iq_almost 8.4% → 1.3%）	301,827	-0.87%
v2.10	jirl 去串行化（serial -38%）	298,664	-1.05%

最终成绩：lean 384,779 → 298,664（-22.4%），SoC 386,839 → 299,861（-22.5%），IPC 0.238 → 0.307（+29%）。

4.1 证伪记录（负结果同样重要）

ROB 32 → 64（两次独立实验证伪）：扩容后 rob_full 停顿 50,161 → 0 完全消除，但瓶颈迁移至 IQ（4.4k → 34.7k 拍）与物理寄存器耗尽（fl 1 → 23.6k 拍），净效果 -0.8%（反而变慢）。结论：**ROB/IQ/PRF 窗口资源必须联动扩容**，单独扩 ROB 无独立收益；IQ 年龄矩阵 $O(n^2)$ 增长有实质频率风险（其选择逻辑本就在最深路径候选中），故维持 ROB32 / IQ16 / PRF64 的均衡点。

四行 refill 提前 resp 优化：后台填期间提前响应第 2 行请求——r_{last} 时刻 预取缓冲装的是第 4 行，提前响应必取错数据，上线即回退。教训：性能优化先过 正确性边界条件推演，再进仿真。

4.2 当前停顿画像（v2.10）

表 3 分派停顿分解（lean 环境，总 298,664 拍）

停顿类型	拍数	占比	说明
前端饥饿	138,030	46%	icache miss（命中率 83%）+ 窗口满传导
ROB 满	50,161	16.8%	窗口资源联动，扩容实验见 §4.1
串行点	12,339	4.1%	csr/ll/sc/syscall/ertn 必需语义
IQ 近满	4,427	1.5%	v2.9 已治理（8.4% → 1.5%）
分支 flush	60 次	—	BPU 精度高，flush 极罕见

5. 频率设计与验证

方法：每版迭代后用 `yosys synth_xilinx`（xc7, -retime, flatten）综合，自研 LUT 深度分析脚本统计全局长组合路径 top15 端点，**零侵蚀才允许保留**。该门禁使"性能优化"与"频率保持"两个目标在迭代中解耦——任何拉长关键路径的改动 在仿真收益确认后仍会被否决。

- 基线（v2.4b）：全局最深 7 级 LUT（lsu store buffer 压缩链、iq 年龄选择）。
- 缓存三件套（ICache/DCache/预取）复测：最深 5 级，零频率代价。
- v2.5 复测：top15 最深 7 级，6 个地址区间比较器零上榜，缓存/前端/预测器零侵蚀。
- v2.10（最终版）复测：见附录 A。

提交频率决策：依据 LUT 深度（最深 7 级）保守取值。`perf_clock.json` 申报 100 MHz 作为首跑基线——赛事时序门禁要求 setup/hold WNS 非负（负值成绩无效），且 CI 会回报实现后 `timing_summary.rpt` 与各项 benchmark 计数明细，首跑数据到手后再决定上调空间。评分公式为 20 项 benchmark 的 GEOMEAN(openLA500 参考周期数 / 本机周期数)，得分主体由 IPC 决定。

6. 正确性验证方法学

1. **双环境回归：**verilator 时序仿真，lean 环境（AXI-SRAM 桥直连）+ 完整 SoC 环境（APB 外设/UART/NAND/CONFREG/AXI 随机延迟），58/58 功能测试点全过是任何提交的硬门槛。

- 2. **定向测试**：ll/sc 原子指令、中断链路（TCFG/TICLR/定时器中断/ertn 恢复）、例外码与 era/badv 精确性共 7 项定向用例。
- 3. **探针体系**：专用 testbench 探针库（停顿分解 / icache 统计 / 前端逐拍 / FIFO 头部 / 例外码 / 提交转储）。所有疑难缺陷（v2.7 FIFO 数据标签错位、v2.8 后台行写错位、freelist 泄漏死锁、ROB64 提交标签截断死锁）均由探针数据定位根因，而非猜测。
- 4. **仿真器交叉**：iverilog 与 verilator 双仿真器行为比对（曾发现索引截断的仿真器实现差异并显式修复，保证 RTL 可移植性）。

7. 参考与声明

- 1. 大赛指定实验平台 Chiplab（nscsc2026 固定快照 68c20a5）提供的 SoC 环境、AXI-SRAM 桥、功能/性能测试程序。
- 2. 《CPU 设计实战：LoongArch 版》（汪文祥、邢金璋），流水线握手与译码组织方式。
- 3. Hennessy & Patterson, *Computer Architecture: A Quantitative Approach*，分支预测、重命名、ROB 等微架构机制。
- 4. openLA500 开源实现的公开文档（机制学习参考）。

全部 RTL 为本团队原创实现；第三方参考仅用于机制学习，无代码级复制。

附录 A. 综合与 LUT 深度报告（v2.10）

综合环境：yosys synth_xilinx -top mycpu_top -family xc7 -retime +flatten，自研脚本对扁平化网表做全局最长组合路径（LUT 级数）统计。

表 A-1 v2.10 全局最长组合路径（yosys xc7 -retime 网表统计）

深度	条数（top15 内）	路径构成	性质
8 级	≥10（top10 全为同构）	u_iq 发射选择 mux 树（MUXF7/MUXF8×6-7）+ u_rob 标签算术（CARRY4×1）	IQ16 oldest-first 发射选择
≤7 级	其余	lsu store buffer 压缩链、iq 年龄比较等（与 v2.5 基线同）	历史既有

A.1 与基线对比及判定

- v2.5 基线（IQ12）：最深 7 级（lsu sb 压缩 / iq 年龄选择）。
- v2.10（IQ16）：最深 8 级（+1），来源明确——IQ 12→16 使发射选择 mux 扇入加倍，MUXF7/F8 树加深一级；ROB 标签 CARRY4 为既有端点。

- **v2.8 icache 四行 refill / v2.10 译码改动零新上榜**：缓存、前端、预测器、译码模块无任何端点进入 top15——迭代主体零频率侵蚀。

判定：接受 8 级，维持 IQ16。理由：(1) 赛事评分公式为周期数比值几何平均，频率不直接进分，100 MHz (10 ns 周期) 下 8 级路径 (含 FPGA 专用快 mux MUXF7/F8，实际延迟优于等效 LUT) 裕量充足；(2) 若为压低 1 级而对发射选择 打拍，发射延迟 +1 拍将直接损失 IPC——以确定性的 IPC 损失换用不到的频率 裕量，违背本设计"频率服务于得分"的权衡原则；(3) perf_clock.json 据此申报 100 MHz 首跑，WNS 门禁由 CI 的 Vivado 实现报告最终把关。

综合环境：yosys 0.53+ `synth_xilinx -top mycpu_top -family xc7 -retime + flatten`；abc 映射耗时 1026 s (IQ16 年龄矩阵为最大模块)。统计脚本对扁平化网表做全局长组合路径 (LUT 级数) DFS 统计。